

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate

7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze
8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I M.sharmila(192572139) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. **Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.**

QUERIES EXPLANATION

You are given multiple tables containing related data. You need to retrieve specific information using JOINS and subqueries.

Main concepts:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- Subqueries using SELECT
- WHERE
- Matching columns between tables

Example:

Suppose we have:

student(student_id, name, dept_id)

department(dept_id, dept_name)

To display students along with their department:

```
SELECT s.name, d.dept_name
```

```
FROM student s
```

```
INNER JOIN department d
```

```
ON s.dept_id = d.dept_id;
```

Explanation:

The INNER JOIN combines the student and department tables using the common dept_id.

Only students having a matching department are displayed.

A subquery can be used to find students whose marks are greater than the average:

```
SELECT name, marks
```

```
FROM student
```

```
WHERE marks > (SELECT AVG(marks) FROM student);
```

Explanation:

1. The inner query calculates the average marks.
2. The outer query compares every student's marks with that average.
3. Students scoring above the average are displayed.

```

mysql> USE college;
Database changed
mysql> CREATE TABLE DEPARTMENT (
  ->     Dept_ID VARCHAR(10) PRIMARY KEY,
  ->     Dept_Name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE STUDENT (
  ->     Student_ID VARCHAR(10) PRIMARY KEY,
  ->     Student_Name VARCHAR(50),
  ->     Dept_ID VARCHAR(10),
  ->     FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TABLE COURSE (
  ->     Course_ID VARCHAR(10) PRIMARY KEY,
  ->     Course_Name VARCHAR(50),
  ->     Dept_ID VARCHAR(10),
  ->     FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)
  -> );
Query OK, 0 rows affected (0.11 sec)

mysql> INSERT INTO DEPARTMENT VALUES
  -> ('D01','CSE'),
  -> ('D02','ECE'),
  -> ('D03','EEE');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO STUDENT VALUES
  -> ('S101','Arun','D01'),
  -> ('S102','Priya','D02'),
  -> ('S103','Kiran','D01'),
  -> ('S104','Meena','D03');
Query OK, 4 rows affected (0.01 sec)

mysql>
mysql> INSERT INTO STUDENT VALUES
  -> ('S101','Arun','D01'),
  -> ('S102','Priya','D02'),
  -> ('S103','Kiran','D01'),
  -> ('S104','Meena','D03');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO COURSE VALUES
  -> ('C101','DBMS','D01'),
  -> ('C102','Operating Systems','D01'),
  -> ('C201','Digital Electronics','D02'),
  -> ('C301','Power Systems','D03');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM DEPARTMENT;
+-----+-----+
| Dept_ID | Dept_Name |
+-----+-----+
| D01     | CSE       |
| D02     | ECE       |
| D03     | EEE       |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM STUDENT;
+-----+-----+-----+
| Student_ID | Student_Name | Dept_ID |
+-----+-----+-----+
| S101       | Arun         | D01     |
| S102       | Priya        | D02     |
| S103       | Kiran        | D01     |
| S104       | Meena        | D03     |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM COURSE;

```

```
mysql> SELECT * FROM COURSE;
```

Course_ID	Course_Name	Dept_ID
C101	DBMS	D01
C102	Operating Systems	D01
C201	Digital Electronics	D02
C301	Power Systems	D03

4 rows in set (0.00 sec)

```
mysql> SELECT s.Student_Name, d.Dept_Name
-> FROM STUDENT s
-> INNER JOIN DEPARTMENT d
-> ON s.Dept_ID = d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Kiran	CSE
Priya	ECE
Meena	EEE

4 rows in set (0.00 sec)

```
mysql> SELECT s.Student_Name, d.Dept_Name
-> FROM STUDENT s
-> LEFT JOIN DEPARTMENT d
-> ON s.Dept_ID = d.Dept_ID;
```

Student_Name	Dept_Name
Arun	CSE
Priya	ECE
Kiran	CSE
Meena	EEE

4 rows in set (0.03 sec)

```
mysql> SELECT Student_Name
```

```
mysql> SELECT Course_Name
-> FROM COURSE
-> WHERE Dept_ID = (
-> SELECT Dept_ID
-> FROM DEPARTMENT
-> WHERE Dept_Name = 'CSE'
-> );
```

Course_Name
DBMS
Operating Systems

2 rows in set (0.00 sec)

```
mysql> SELECT Dept_Name
-> FROM DEPARTMENT
-> WHERE Dept_ID IN (
-> SELECT Dept_ID
-> FROM COURSE
-> );
```

Dept_Name
CSE
ECE
EEE

3 rows in set (0.00 sec)

```
mysql> SELECT s.Student_Name,
->         d.Dept_Name,
->         c.Course_Name
-> FROM STUDENT s
-> JOIN DEPARTMENT d
-> ON s.Dept_ID = d.Dept_ID
-> JOIN COURSE c
-> ON d.Dept_ID = c.Dept_ID;
```

Student_Name	Dept_Name	Course_Name
Arun	CSE	DBMS
Arun	CSE	Operating Systems
Kiran	CSE	DBMS
Kiran	CSE	Operating Systems
Priya	ECE	Digital Electronics
Meena	EEE	Power Systems

```
6 rows in set (0.00 sec)

mysql>
```

2. **Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.**

QUERIES EXPLANTION

Relations

- Student(StudentID, Name, Department)
- Marks(StudentID, Subject, Marks)

SQL Query

```
SELECT Student.Name
FROM Student
INNER JOIN Marks
```

ON Student.StudentID = Marks.StudentID

WHERE Student.Department = 'CSE'

AND Marks.Marks > 80;

Step 1: Join Operation

Student ⋈ Student.StudentID = Marks.StudentID Marks

Step 2: Selection Operation σ Department = 'CSE' $\wedge$ Marks > 80

(Student ⋈ Student.StudentID = Marks.StudentID Marks)

Step 3: Projection Operation π

Name

(

σ Department = 'CSE' $\wedge$ Marks > 80

(Student ⋈ Student.StudentID = Marks.StudentID Marks)

)

Final Relational Algebra Expression π Name (σ

Department = 'CSE' $\wedge$ Marks > 80

(Student ⋈ Student.StudentID = Marks.StudentID Marks)) **Result**

The required records satisfying multiple conditions across the Student and Marks relations are retrieved successfully using the Relational Algebra operators Selection (σ), Projection (π), and Join ($\bowtie$).

This is the proper line-by-line format that is typically expected in a DBMS practical record.

```
mysql> SELECT s.Student_Name,
->         d.Dept_Name,
->         c.Course_Name
-> FROM STUDENT s
-> JOIN DEPARTMENT d
-> ON s.Dept_ID = d.Dept_ID
-> JOIN COURSE c
-> ON d.Dept_ID = c.Dept_ID;
```

Student_Name	Dept_Name	Course_Name
Arun	CSE	DBMS
Arun	CSE	Operating Systems
Kiran	CSE	DBMS
Kiran	CSE	Operating Systems
Priya	ECE	Digital Electronics
Meena	EEE	Power Systems

```
6 rows in set (0.00 sec)

mysql>
```

3. Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.

QUERIES EXPLANATION

For a student system, you could create:

STUDENT(Student_ID, Name, Dept_ID, Email)

DEPARTMENT(Dept_ID, Dept_Name)

COURSE(Course_ID, Course_Name, Credits)

ENROLLMENT(Student_ID, Course_ID, Marks)

Explanation of the design:

- Student_ID uniquely identifies a student.
- Dept_ID connects students with departments.
- Course_ID identifies each course.
- ENROLLMENT connects students and courses.
- Marks stores the student's performance.

Example query:

```
SELECT s.name, e.marks
```

```
FROM student s
```

```
JOIN enrollment e
```

```
ON s.student_id = e.student_id
```

```
WHERE e.marks > 80;
```

This retrieves students who scored more than 80.

An update query could be:

```
UPDATE student
```

```
SET email = 'student@gmail.com'
```

```
WHERE student_id = 101;
```

Explanation:

The UPDATE command modifies an existing record. The WHERE clause ensures that only student 101 is modified.


```

mysql> CREATE DATABASE CollegeDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE CollegeDB;
Database changed
mysql>
mysql> CREATE TABLE DEPARTMENT(
->     Dept_ID VARCHAR(5) PRIMARY KEY,
->     Dept_Name VARCHAR(50),
->     HOD VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE STUDENT(
->     Reg_No VARCHAR(10) PRIMARY KEY,
->     Student_Name VARCHAR(50),
->     Gender CHAR(1),
->     Age INT,
->     Dept_ID VARCHAR(5),
->     FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TABLE FACULTY(
->     Faculty_ID VARCHAR(5) PRIMARY KEY,
->     Faculty_Name VARCHAR(50),
->     Designation VARCHAR(30),
->     Age INT,
->     Dept_ID VARCHAR(5),
->     FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE COURSE(
->     Course_ID VARCHAR(5) PRIMARY KEY,
->     Course_Name VARCHAR(50),
->     Credits INT,

```

```

->     Faculty_ID VARCHAR(5),
->     FOREIGN KEY (Faculty_ID) REFERENCES FACULTY(Faculty_ID)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO DEPARTMENT VALUES
-> ('D001','Computer Science','Dr.Kumar'),
-> ('D002','Information Technology','Dr.Priya'),
-> ('D003','Electronics','Dr.Ravi'),
-> ('D004','Mechanical','Dr.Suresh'),
-> ('D005','Civil','Dr.Meena');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO FACULTY VALUES
-> ('F001','Mohan','Professor',58,'D001'),
-> ('F002','Revathi','Assistant Professor',45,'D002'),
-> ('F003','Ramesh','Professor',54,'D003'),
-> ('F004','Lakshmi','Associate Professor',49,'D004'),
-> ('F005','Anitha','Professor',52,'D005');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO STUDENT VALUES
-> ('191711342','Arun','M',20,'D001'),
-> ('191711343','Divya','F',19,'D002'),
-> ('191711344','Rahul','M',21,'D001'),
-> ('191711345','Priya','F',20,'D003'),
-> ('191711346','Karthik','M',22,'D004');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO COURSE VALUES
-> ('C101','Database Management System',4,'F001'),
-> ('C102','Operating System',3,'F001'),
-> ('C103','Computer Networks',4,'F002'),
-> ('C104','Digital Electronics',4,'F003'),

```

```
-> ('C105','Engineering Mechanics',3,'F004');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

```
mysql>
```

```
mysql> SELECT * FROM DEPARTMENT;
```

Dept_ID	Dept_Name	HOD
D001	Computer Science	Dr.Kumar
D002	Information Technology	Dr.Priya
D003	Electronics	Dr.Ravi
D004	Mechanical	Dr.Suresh
D005	Civil	Dr.Meena

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM STUDENT;
```

Reg_No	Student_Name	Gender	Age	Dept_ID
191711342	Arun	M	20	D001
191711343	Divya	F	19	D002
191711344	Rahul	M	21	D001
191711345	Priya	F	20	D003
191711346	Karthik	M	22	D004

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM FACULTY;
```

Faculty_ID	Faculty_Name	Designation	Age	Dept_ID
F001	Mohan	Professor	58	D001
F002	Revathi	Assistant Professor	45	D002
F003	Ramesh	Professor	54	D003
F004	Lakshmi	Associate Professor	49	D004
F005	Anitha	Professor	52	D005

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM COURSE;
```

Course_ID	Course_Name	Credits	Faculty_ID
C101	Database Management System	4	F001
C102	Operating System	3	F001
C103	Computer Networks	4	F002
C104	Digital Electronics	4	F003
C105	Engineering Mechanics	3	F004

```
5 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> SELECT * FROM STUDENT WHERE Dept_ID='D001';
```

Reg_No	Student_Name	Gender	Age	Dept_ID
191711342	Arun	M	20	D001
191711344	Rahul	M	21	D001

```
2 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> SELECT S.Reg_No,S.Student_Name,D.Dept_Name
-> FROM STUDENT S
-> INNER JOIN DEPARTMENT D
-> ON S.Dept_ID=D.Dept_ID;
```

Reg_No	Student_Name	Dept_Name
191711342	Arun	Computer Science
191711343	Divya	Information Technology
191711344	Rahul	Computer Science
191711345	Priya	Electronics
191711346	Karthik	Mechanical

```
5 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> UPDATE STUDENT
```

```
mysql>
mysql> UPDATE STUDENT
    -> SET Age=22
    -> WHERE Reg_No='191711342';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql>
mysql> UPDATE FACULTY
    -> SET Age=59
    -> WHERE Faculty_ID='F001';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql>
mysql> UPDATE COURSE
    -> SET Credits=5
    -> WHERE Course_ID='C101';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql>
mysql> DELETE FROM STUDENT
    -> WHERE Reg_No='191711346';
Query OK, 1 row affected (0.01 sec)
```

```
mysql>
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+-----+-----+
| Reg_No | Student_Name | Gender | Age | Dept_ID |
+-----+-----+-----+-----+-----+
| 191711342 | Arun | M | 22 | D001 |
| 191711343 | Divya | F | 19 | D002 |
| 191711344 | Rahul | M | 21 | D001 |
| 191711345 | Priya | F | 20 | D003 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> DELETE FROM STUDENT
    -> WHERE Reg_No='191711346';
Query OK, 1 row affected (0.01 sec)
```

```
mysql>
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+-----+-----+
| Reg_No | Student_Name | Gender | Age | Dept_ID |
+-----+-----+-----+-----+-----+
| 191711342 | Arun | M | 22 | D001 |
| 191711343 | Divya | F | 19 | D002 |
| 191711344 | Rahul | M | 21 | D001 |
| 191711345 | Priya | F | 20 | D003 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM FACULTY;
+-----+-----+-----+-----+-----+
| Faculty_ID | Faculty_Name | Designation | Age | Dept_ID |
+-----+-----+-----+-----+-----+
| F001 | Mohan | Professor | 59 | D001 |
| F002 | Revathi | Assistant Professor | 45 | D002 |
| F003 | Ramesh | Professor | 54 | D003 |
| F004 | Lakshmi | Associate Professor | 49 | D004 |
| F005 | Anitha | Professor | 52 | D005 |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM COURSE;
+-----+-----+-----+-----+
| Course_ID | Course_Name | Credits | Faculty_ID |
+-----+-----+-----+-----+
| C101 | Database Management System | 5 | F001 |
| C102 | Operating System | 3 | F001 |
| C103 | Computer Networks | 4 | F002 |
| C104 | Digital Electronics | 4 | F003 |
| C105 | Engineering Mechanics | 3 | F004 |
+-----+-----+-----+-----+
5 rows in set (0.05 sec)
```

4. Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.

QUERIES EXPLANATION

Example:

```
SELECT name, marks
FROM student
WHERE marks > (
    SELECT AVG(marks)
    FROM student
);
```

Execution process:

Step 1: Execute the inner query:

```
SELECT AVG(marks)
FROM student;
```

Suppose the result is:

75

Step 2: The original query becomes conceptually:

```
SELECT name, marks
FROM student
WHERE marks > 75;
```

Step 3: Every student's marks are compared with 75.

Step 4: Only students scoring above 75 are returned.

Therefore, when analyzing nested queries, you should explain:

1. Which query executes first.
2. What result it produces.
3. How that result is used by the outer query.
4. What final rows are returned.

```
mysql> CREATE DATABASE COLLEGE;
Query OK, 1 row affected (0.03 sec)

mysql> USE COLLEGE;
Database changed
mysql>
mysql> CREATE TABLE STUDENT
-> (
->     STUDENT_ID INT PRIMARY KEY,
->     STUDENT_NAME VARCHAR(50),
->     DEPARTMENT VARCHAR(20),
->     MARKS INT
-> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO STUDENT VALUES
-> (101,'Arun','CSE',85),
-> (102,'Bala','ECE',78),
-> (103,'Charan','CSE',92),
-> (104,'Divya','IT',88),
-> (105,'Elan','CSE',75);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+-----+
| STUDENT_ID | STUDENT_NAME | DEPARTMENT | MARKS |
+-----+-----+-----+-----+
| 101 | Arun | CSE | 85 |
| 102 | Bala | ECE | 78 |
| 103 | Charan | CSE | 92 |
| 104 | Divya | IT | 88 |
| 105 | Elan | CSE | 75 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql>
mysql> SELECT AVG(MARKS) AS CSE_AVERAGE
-> FROM STUDENT
-> WHERE DEPARTMENT='CSE';
+-----+
| CSE_AVERAGE |
+-----+
| 84.0000 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT STUDENT_NAME, MARKS
-> FROM STUDENT
-> WHERE MARKS >
-> (
->     SELECT AVG(MARKS)
->     FROM STUDENT
->     WHERE DEPARTMENT='CSE'
-> );
+-----+-----+
| STUDENT_NAME | MARKS |
+-----+-----+
| Arun | 85 |
| Charan | 92 |
| Divya | 88 |
+-----+-----+
3 rows in set (0.03 sec)
```

5.Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency

QUERIES EXPLANATION

For example, find students belonging to the CSE department.

Using a subquery:

```
SELECT name
FROM student
WHERE dept_id = (
    SELECT dept_id
    FROM department
    WHERE dept_name = 'CSE'
);
```

Using a join:

```
SELECT s.name
FROM student s
JOIN department d
ON s.dept_id = d.dept_id
WHERE d.dept_name = 'CSE';
```

Evaluation:

You should not simply say that one is always faster. Performance depends on the database engine, indexes, table sizes, and query structure.

```

mysql> CREATE DATABASE COLLEGE;
Query OK, 1 row affected (0.02 sec)

mysql> USE COLLEGE;
Database changed
mysql>
mysql> CREATE TABLE DEPARTMENT
-> (
->     DEPT_ID INT PRIMARY KEY,
->     DEPT_NAME VARCHAR(30)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TABLE STUDENT
-> (
->     STUDENT_ID INT PRIMARY KEY,
->     STUDENT_NAME VARCHAR(30),
->     DEPT_ID INT,
->     MARKS INT,
->     FOREIGN KEY (DEPT_ID) REFERENCES DEPARTMENT(DEPT_ID)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> INSERT INTO DEPARTMENT VALUES
-> (1,'CSE'),
-> (2,'ECE'),
-> (3,'IT');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO STUDENT VALUES
-> (101,'Arun',1,85),
-> (102,'Bala',2,78),
-> (103,'Charan',1,92),
-> (104,'Divya',3,88),
-> (105,'Elan',1,75);
Query OK, 5 rows affected (0.01 sec)

```

```

-> (105,'Elan',1,75);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> -- USING JOIN
mysql> SELECT S.STUDENT_NAME, D.DEPT_NAME, S.MARKS
-> FROM STUDENT S
-> INNER JOIN DEPARTMENT D
-> ON S.DEPT_ID = D.DEPT_ID;
+-----+-----+-----+
| STUDENT_NAME | DEPT_NAME | MARKS |
+-----+-----+-----+
| Arun         | CSE       | 85    |
| Charan       | CSE       | 92    |
| Elan         | CSE       | 75    |
| Bala         | ECE       | 78    |
| Divya        | IT        | 88    |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> -- USING SUBQUERY
mysql> SELECT STUDENT_NAME,
->     (SELECT DEPT_NAME
->      FROM DEPARTMENT
->      WHERE DEPT_ID = STUDENT.DEPT_ID) AS DEPARTMENT,
->     MARKS
-> FROM STUDENT;
+-----+-----+-----+
| STUDENT_NAME | DEPARTMENT | MARKS |
+-----+-----+-----+
| Arun         | CSE       | 85    |
| Bala         | ECE       | 78    |
| Charan       | CSE       | 92    |
| Divya        | IT        | 88    |
| Elan         | CSE       | 75    |
+-----+-----+-----+
5 rows in set (0.00 sec)

```

6.Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

QUERIES EXPLANATION

Suppose:

STUDENT(Student_ID, Name, Email, Phone, Marks)

You don't want every user to see phone numbers.

Create:

```
CREATE VIEW student_public AS  
SELECT student_id, name, email, marks  
FROM student;
```

Then:

```
SELECT *  
FROM student_public;
```

Explanation:

The view displays only selected columns.

Why use views?

Security:

Sensitive columns can be hidden.

Abstraction:

Users don't need to know the underlying table structure.

Simplicity:

Complex queries can be stored as a view and reused.

For example:

```
CREATE VIEW top_students AS  
SELECT student_id, name, marks  
FROM student  
WHERE marks >= 80;
```

Now users can simply execute:

```
SELECT * FROM top_students;
```



```

mysql> DROP DATABASE IF EXISTS COLLEGE;
Query OK, 2 rows affected (0.07 sec)

mysql> CREATE DATABASE COLLEGE;
Query OK, 1 row affected (0.01 sec)

mysql> USE COLLEGE;
Database changed
mysql>
mysql> CREATE TABLE STUDENT
-> (
->     STUDENT_ID INT PRIMARY KEY,
->     STUDENT_NAME VARCHAR(50),
->     DEPARTMENT VARCHAR(20),
->     MARKS INT,
->     EMAIL VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO STUDENT VALUES
-> (101, 'Arun', 'CSE', 85, 'arun@gmail.com'),
-> (102, 'Bala', 'ECE', 78, 'bala@gmail.com'),
-> (103, 'Charan', 'CSE', 92, 'charan@gmail.com'),
-> (104, 'Divya', 'IT', 88, 'divya@gmail.com'),
-> (105, 'Elan', 'CSE', 75, 'elan@gmail.com');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

```

```

mysql>
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+-----+-----+
| STUDENT_ID | STUDENT_NAME | DEPARTMENT | MARKS | EMAIL |
+-----+-----+-----+-----+-----+
| 101 | Arun | CSE | 85 | arun@gmail.com |
| 102 | Bala | ECE | 78 | bala@gmail.com |
| 103 | Charan | CSE | 92 | charan@gmail.com |
| 104 | Divya | IT | 88 | divya@gmail.com |
| 105 | Elan | CSE | 75 | elan@gmail.com |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> CREATE VIEW STUDENT_VIEW AS
-> SELECT
->     STUDENT_ID,
->     STUDENT_NAME,
->     DEPARTMENT,
->     MARKS
-> FROM STUDENT;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SELECT * FROM STUDENT_VIEW;
+-----+-----+-----+-----+
| STUDENT_ID | STUDENT_NAME | DEPARTMENT | MARKS |
+-----+-----+-----+-----+
| 101 | Arun | CSE | 85 |
| 102 | Bala | ECE | 78 |
| 103 | Charan | CSE | 92 |
| 104 | Divya | IT | 88 |
| 105 | Elan | CSE | 75 |
+-----+-----+-----+-----+
5 rows in set (0.03 sec)

```

7. Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.

Relations

Student(StudentID, Name, Department)

Marks(StudentID, Subject, Marks)

Query

Retrieve the names of students who belong to the CSE department and scored more than 80 marks.

Relational Algebra Expression π

Name

(

σ Department='CSE' $\wedge$ Marks>80

(

Student $\bowtie$ Student.StudentID = Marks.StudentID Marks

)

)

Equivalent SQL Query

SELECT Student.Name

FROM Student

INNER JOIN Marks

ON Student.StudentID = Marks.StudentID

WHERE Student.Department = 'CSE'

AND Marks.Marks > 80;

Difference Between Relational

Algebra and SQL

Relational Algebra	SQL
Uses symbols such as π , σ , and $\bowtie$	Uses keywords like SELECT, FROM, WHERE, JOIN
Procedural query language	Declarative query language
Describes how to retrieve data	Describes what data to retrieve
Used for theoretical representation	Used for implementation in DBMS

RESULT

The Relational Algebra expression was successfully converted into an equivalent SQL query. Both produce the same output, but Relational Algebra uses mathematical notation while SQL uses English-like commands.

8.Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.

Suppose the query is:

```
SELECT *  
FROM student  
WHERE name IN (  
    SELECT name  
    FROM student  
    WHERE marks > 80  
);
```

If the intention is simply to find students with marks above 80, this is unnecessarily complicated.

Better:

```
SELECT name, marks  
FROM student  
WHERE marks > 80;
```

You can also use indexes.

For example:

```
CREATE INDEX idx_student_marks  
ON student(marks);
```

Explanation:

An index can help the DBMS locate rows efficiently when a column is frequently used for searching, filtering, or joining.

Another optimization technique is selecting only required columns:

Instead of:

```
SELECT *
```

```
FROM student;
```

use:

```
SELECT student_id, name  
FROM student;
```

This reduces unnecessary data retrieval.

Important:

You should evaluate optimization by considering:

- Number of rows
- Indexes
- Join conditions
- Filtering
- Amount of data returned
- Execution plan

```
mysql> CREATE DATABASE COLLEGE;
Query OK, 1 row affected (0.01 sec)

mysql> USE COLLEGE;
Database changed
mysql>
mysql> CREATE TABLE DEPARTMENT
-> (
->     DEPT_ID INT PRIMARY KEY,
->     DEPT_NAME VARCHAR(30)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TABLE STUDENT
-> (
->     STUDENT_ID INT PRIMARY KEY,
->     STUDENT_NAME VARCHAR(30),
->     DEPT_ID INT,
->     MARKS INT,
->     INDEX IDX_DEPT(DEPT_ID),
->     FOREIGN KEY (DEPT_ID) REFERENCES DEPARTMENT(DEPT_ID)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO DEPARTMENT VALUES
-> (1,'CSE'),
-> (2,'ECE'),
-> (3,'IT');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO STUDENT VALUES
-> (101,'Arun',1,85),
-> (102,'Bala',2,78),
-> (103,'Charan',1,92),
-> (104,'Divya',3,88),
-> (105,'Elan',1,75);
```

```

-> (105,'Elan',1,75);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Inefficient Query (Using Subquery)
mysql> SELECT STUDENT_NAME
-> FROM STUDENT
-> WHERE DEPT_ID =
-> (
->     SELECT DEPT_ID
->     FROM DEPARTMENT
->     WHERE DEPT_NAME='CSE'
-> );
+-----+
| STUDENT_NAME |
+-----+
| Arun         |
| Charan       |
| Elan         |
+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Optimized Query (Using JOIN)
mysql> SELECT S.STUDENT_NAME, D.DEPT_NAME
-> FROM STUDENT S
-> INNER JOIN DEPARTMENT D
-> ON S.DEPT_ID = D.DEPT_ID
-> WHERE D.DEPT_NAME='CSE';
+-----+-----+
| STUDENT_NAME | DEPT_NAME |
+-----+-----+
| Arun         | CSE       |
| Charan       | CSE       |
| Elan         | CSE       |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Create View

```

3 rows in set (0.00 sec)

mysql>

mysql> -- Optimized Query (Using JOIN)

mysql> SELECT S.STUDENT_NAME, D.DEPT_NAME

-> FROM STUDENT S

-> INNER JOIN DEPARTMENT D

-> ON S.DEPT_ID = D.DEPT_ID

-> WHERE D.DEPT_NAME='CSE';

STUDENT_NAME	DEPT_NAME
Arun	CSE
Charan	CSE
Elan	CSE

3 rows in set (0.00 sec)

mysql>

mysql> -- Create View

mysql> CREATE VIEW CSE_STUDENTS AS

-> SELECT S.STUDENT_NAME, D.DEPT_NAME, S.MARKS

-> FROM STUDENT S

-> INNER JOIN DEPARTMENT D

-> ON S.DEPT_ID = D.DEPT_ID

-> WHERE D.DEPT_NAME='CSE';

Query OK, 0 rows affected (0.01 sec)

mysql>

mysql> -- Display View

mysql> SELECT * FROM CSE_STUDENTS;

STUDENT_NAME	DEPT_NAME	MARKS
Arun	CSE	85
Charan	CSE	92
Elan	CSE	75

3 rows in set (0.00 sec)

mysql>

9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.

QUERIES EXPLANATION

- GROUP BY
- Aggregate functions
- Nested queries

Example problem:

Find departments whose average student marks are greater than the overall average marks.

Query:

```
SELECT dept_id, AVG(marks) AS average_marks
FROM student
GROUP BY dept_id
HAVING AVG(marks) > (
    SELECT AVG(marks)
    FROM student
)
;
```

Explanation:

Step 1:

```
SELECT AVG(marks)
FROM student;
```

calculates the overall average.

Step 2:

```
GROUP BY dept_id
```

divides students into groups according to their department.

Step 3:

```
AVG(marks)
```

calculates the average marks for each department.

Step 4:

```
HAVING AVG(marks) > (...)
```

keeps only departments whose average is greater than the overall average.

This is a complex query because it combines aggregation + grouping + nested subquery.


```

mysql> CREATE DATABASE COLLEGE;
Query OK, 1 row affected (0.01 sec)

mysql> USE COLLEGE;
Database changed
mysql>
mysql> CREATE TABLE DEPARTMENT
-> (
->     DEPT_ID INT PRIMARY KEY,
->     DEPT_NAME VARCHAR(30)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TABLE STUDENT
-> (
->     STUDENT_ID INT PRIMARY KEY,
->     STUDENT_NAME VARCHAR(30),
->     DEPT_ID INT,
->     MARKS INT,
->     FOREIGN KEY (DEPT_ID) REFERENCES DEPARTMENT(DEPT_ID)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> INSERT INTO DEPARTMENT VALUES
-> (1, 'CSE'),
-> (2, 'ECE'),
-> (3, 'IT');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO STUDENT VALUES
-> (101, 'Arun', 1, 85),
-> (102, 'Bala', 2, 78),
-> (103, 'Charan', 1, 92),
-> (104, 'Divya', 3, 88),
-> (105, 'Elan', 1, 75),
-> (106, 'Farhan', 2, 82),
-> (107, 'Gokul', 3, 95);

```

```

-> (106, 'Farhan', 2, 82),
-> (107, 'Gokul', 3, 95);
Query OK, 7 rows affected (0.03 sec)
Records: 7  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM DEPARTMENT;
+-----+-----+
| DEPT_ID | DEPT_NAME |
+-----+-----+
| 1 | CSE |
| 2 | ECE |
| 3 | IT |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+-----+
| STUDENT_ID | STUDENT_NAME | DEPT_ID | MARKS |
+-----+-----+-----+-----+
| 101 | Arun | 1 | 85 |
| 102 | Bala | 2 | 78 |
| 103 | Charan | 1 | 92 |
| 104 | Divya | 3 | 88 |
| 105 | Elan | 1 | 75 |
| 106 | Farhan | 2 | 82 |
| 107 | Gokul | 3 | 95 |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)

mysql>
mysql> SELECT D.DEPT_NAME,
->     COUNT(S.STUDENT_ID) AS TOTAL_STUDENTS,
->     AVG(S.MARKS) AS AVERAGE_MARKS
-> FROM STUDENT S
-> JOIN DEPARTMENT D
-> ON S.DEPT_ID = D.DEPT_ID
-> GROUP BY D.DEPT_NAME
-> HAVING AVG(S.MARKS) >
-> (
->     SELECT AVG(MARKS)

```

```
mysql> SELECT * FROM STUDENT;
```

STUDENT_ID	STUDENT_NAME	DEPT_ID	MARKS
101	Arun	1	85
102	Bala	2	78
103	Charan	1	92
104	Divya	3	88
105	Elan	1	75
106	Farhan	2	82
107	Gokul	3	95

```
7 rows in set (0.00 sec)
```

```
mysql>
```

```
mysql> SELECT D.DEPT_NAME,  
-> COUNT(S.STUDENT_ID) AS TOTAL_STUDENTS,  
-> AVG(S.MARKS) AS AVERAGE_MARKS  
-> FROM STUDENT S  
-> JOIN DEPARTMENT D  
-> ON S.DEPT_ID = D.DEPT_ID  
-> GROUP BY D.DEPT_NAME  
-> HAVING AVG(S.MARKS) >  
-> (  
-> SELECT AVG(MARKS)  
-> FROM STUDENT  
-> );
```

DEPT_NAME	TOTAL_STUDENTS	AVERAGE_MARKS
IT	2	91.5000

```
1 row in set (0.01 sec)
```

10. Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.

QUERIES EXPLANATION

Use JOIN when:

You need information from multiple related tables.

Example:

```
SELECT s.name, d.dept_name  
FROM student s  
JOIN department d  
ON s.dept_id = d.dept_id;
```

Reason:

Student and department information must be combined.

Use SUBQUERY when:

The result of one query is required by another query.

Example:

```
SELECT name, marks  
FROM student  
WHERE marks > (  
    SELECT AVG(marks)  
    FROM student  
);
```

Reason:

The outer query needs the average calculated by the inner query.

Use VIEW when:

The same query/result needs to be accessed repeatedly or when you want to restrict access.

```
CREATE VIEW student_details AS  
SELECT student_id, name, dept_id  
FROM student;
```

Reason:

Users can access the required information without directly accessing the complete underlying table.

Quick decision table

Requirement	Best choice	Reason
Combine related tables	JOIN	Connects rows from multiple tables
Compare against another query's result	Subquery	Inner result is used by outer query
Reuse a complex query	VIEW	Stores query definition for reuse
Hide sensitive columns	VIEW	Provides restricted access
Retrieve related data from many tables	JOIN	Efficient relational combination
Find values based on calculated result	Subquery	Useful for comparisons

```

mysql> CREATE DATABASE COLLEGE;
Query OK, 1 row affected (0.02 sec)

mysql> USE COLLEGE;
Database changed
mysql>
mysql> CREATE TABLE DEPARTMENT
-> (
->     DEPT_ID INT PRIMARY KEY,
->     DEPT_NAME VARCHAR(30)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TABLE STUDENT
-> (
->     STUDENT_ID INT PRIMARY KEY,
->     STUDENT_NAME VARCHAR(30),
->     DEPT_ID INT,
->     MARKS INT,
->     FOREIGN KEY (DEPT_ID) REFERENCES DEPARTMENT(DEPT_ID)
-> );
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> INSERT INTO DEPARTMENT VALUES
-> (1,'CSE'),
-> (2,'ECE'),
-> (3,'IT');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO STUDENT VALUES
-> (101,'Arun',1,85),
-> (102,'Bala',2,78),
-> (103,'Charan',1,92),
-> (104,'Divya',3,88),
-> (105,'Elan',1,75);
Query OK, 5 rows affected (0.01 sec)

```

```

mysql>
mysql> CREATE VIEW STUDENT_DETAILS AS
-> SELECT S.STUDENT_ID,
->        S.STUDENT_NAME,
->        D.DEPT_NAME,
->        S.MARKS
-> FROM STUDENT S
-> JOIN DEPARTMENT D
-> ON S.DEPT_ID = D.DEPT_ID;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> SELECT * FROM STUDENT_DETAILS;
+-----+-----+-----+-----+
| STUDENT_ID | STUDENT_NAME | DEPT_NAME | MARKS |
+-----+-----+-----+-----+
| 101 | Arun | CSE | 85 |
| 103 | Charan | CSE | 92 |
| 105 | Elan | CSE | 75 |
| 102 | Bala | ECE | 78 |
| 104 | Divya | IT | 88 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> SELECT STUDENT_NAME, MARKS
-> FROM STUDENT
-> WHERE MARKS >
-> (
->   SELECT AVG(MARKS)
->   FROM STUDENT
-> );
+-----+-----+
| STUDENT_NAME | MARKS |
+-----+-----+
| Arun | 85 |
| Charan | 92 |
| Divya | 88 |
+-----+-----+
3 rows in set (0.00 sec)

```